

Documentación técnica

Arquitectura, datos, metas, sitio web y despliegue
Bodymetria · 12/07/2026

Proyecto de dos piezas: una **app Android nativa** para registrar la vida saludable (captura manual, gráficas y métricas estadísticas; todo local, sin servidor) y un **sitio web** complementario (base de alimentos, recetas y guías). Este documento describe QUÉ se construyó y POR QUÉ; el `manual-aprendizaje.pdf` explica las tecnologías desde cero y `aprendizajes-proyecto.pdf` recoge lo que aprendimos en el camino.

1. Principios de diseño

- **Ligera:** APK ~17 MB, sin librerías de gráficas ni frameworks de inyección; solo Compose, Room, DataStore y `kotlinx.serialization`.
- **Privada:** cero permisos, cero red. Los datos viven en el sandbox de la app (SQLite) y solo salen si el usuario exporta su respaldo JSON.
- **Sin cuentas:** no hay servidor; el respaldo JSON es la portabilidad.
- El identificador del paquete es `com.vidasana` y NO debe cambiarse (Android lo usa como identidad de la app: cambiarlo crearía una app distinta y los usuarios perderían sus datos). El nombre visible es Bodymetria.

2. Arquitectura (tres capas)

1. **Datos** (`app/src/main/java/com/vidasana/datos/`): Room (SQLite) para los registros, DataStore Preferences para el perfil, `kotlinx.serialization` para el respaldo. Sin acceso a red. 2. **Lógica** (`datos/Metas.kt`): funciones puras que calculan metas nutricionales y de agua a partir del perfil y los datos registrados. 3. **UI** (`ui/`): Jetpack Compose + Navigation. Sin ViewModels: las pantallas coleccionan Flows de los DAOs directamente (`collectAsState`) y escriben con corrutinas (`rememberCoroutineScope`). Para el tamaño de esta app, esa capa extra no pagaba su costo.

3. Modelo de datos (Room, esquema v3)

Las fechas se guardan como texto ISO `yyyy-MM-dd`, que ordena correctamente como cadena y evita problemas de zonas horarias.

- `macros`(`fecha` PK, `proteinasG`, `carbohidratosG`, `grasasG`) — las kcal se calculan (4/4/9), no se guardan.
- `sesiones`(`id`, `fecha`, `disciplina`, `duracionMin`, `gastoKcal`, `esfuerzo`, `notas`) -> `ejercicios`(`id`, `sesionId` FK, `nombre`, `orden`) -> `series`(`id`, `ejercicioId` FK, `repeticiones`, `pesoKg?`, `orden`). Borrado en cascada.
- `composicion`(`fecha` PK, `pesoKg`, `musculoKg?`, `grasaPct?`).
- `sueno`(`fecha` PK, `horaDormir`, `horaDespertar`, `calificacion`) — la duración se calcula y soporta cruzar medianoche.
- `diario`(`fecha`, `tipo`) PK compuesta, `valor`, `texto` — LA TABLA COMODÍN: unifica toda sección de "un valor por día". El campo `tipo` distingue: - `estres`, `agua`, `lectura`, `meditacion`, `animo`, `regla` — secciones fijas (`lectura` usa `texto` para el libro). - `micro.<clave>` — micronutrientes (p. ej. `micro.fibra`). - `med.<id>` — métricas médicas personalizadas. Añadir una sección nueva de este estilo no requiere migración.

- `uso_apps(fecha, app)` PK compuesta, minutos.
- `metricas_medicas(id, nombre, tipo, unidad)` — definiciones de la sección Doctor; tipo en {numero, bool, escala}.

Migraciones: v1->v2 añadió `diario.texto`; v2->v3 creó `metricas_medicas`. Regla de oro: al tocar el esquema, escribir SIEMPRE una Migration (nunca `fallbackToDestructiveMigration`, que borra los datos del usuario).

4. Perfil (DataStore)

`Perfil(edad, sexo, estaturaCm, objetivos, configurado, seccionMedica)` en `DataStore` Preferences. Los objetivos son chips multi-selección: Longevidad, Ganancia de músculo, Pérdida de grasa, Escuela, Trabajo, Relaciones personales, Introspección, Lectura/aprendizaje. `seccionMedica` activa la sección Doctor. Si `configurado` es falso, la portada dirige a Configuración.

5. Motor de metas (`Metas.kt`)

- **BMR** por Mifflin-St Jeor: $10 \cdot \text{peso} + 6.25 \cdot \text{estatura} - 5 \cdot \text{edad}$ (+5 hombres, -161 mujeres, -78 promedio para "otro").
- **Gasto diario** = BMR x 1.3 (vida cotidiana ligera) + media diaria REAL de `gastoKcal` de las sesiones de ejercicio de los últimos 7 días. Así el gasto por actividad sale de los datos del usuario, no de un factor inventado.
- **Ajuste por objetivo** (prioridad: pérdida > ganancia > longevidad): - Pérdida de grasa: `kcal` x 0.80, proteína 2.0 g/kg, grasa 25% `kcal`. - Ganancia de músculo: `kcal` x 1.10, proteína 1.8 g/kg, grasa 25% `kcal`. - Longevidad: `kcal` x 0.92 (déficit ligero, <10%), proteína 1.2 g/kg CON ÉNFASIS EN ORIGEN VEGETAL, grasa 30% `kcal`, y nota de "mucha verdura". - Mantenimiento: `kcal` x 1.0, proteína 1.4 g/kg, grasa 30%. Los carbohidratos son el resto de las `kcal`.
- **Agua:** 35 ml por kg de peso.
- **Micros:** metas de referencia (RDA/AI adulto, NIH) por sexo para fibra, calcio, hierro, potasio, sodio (como TOPE, no mínimo), vitaminas C, D y B12. El usuario registra el total diario (el desglose por alimento se calculará en el sitio web); la app marca cumplida/no cumplida cada meta.

6. Estadísticas y gráficas

- `PanelEstadisticas` (en todas las secciones): media de 7 y 30 días, desviación estándar y varianza poblacional (ventana de 30 días, requiere ≥ 2 datos) y tendencia (media 7d vs los 7d previos, con $\sqrt{v}$).
- Gráficas propias con Canvas de Compose (sin librerías): `GraficaLineas` para medidas y niveles, `GraficaBarras` (desde cero) para magnitudes; últimos 30 días, una sola serie en el color primario del tema, rejilla discreta y etiqueta directa del último valor.

7. Pantallas y navegación

Rutas: inicio, config, macros, micros, ejercicio, composicion, sueno, usoCelular, doctor, consejos y diario/{tipo} (pantalla genérica parametrizada para estrés/agua/lectura/meditación/ánimo/regla).

- **Portada:** rejilla 2xN con el resumen del día por sección. El ORDEN depende de los objetivos del perfil (PESOS_OBJETIVO: cada objetivo suma puntos a sus secciones; se ordena por puntaje con desempate por orden base). La tarjeta con dato registrado hoy pinta su icono en color primario. "Ciclo" solo aparece con perfil femenino; "Doctor" solo si se activó en Configuración.
- **Ejercicio:** disciplina con sugerencias (catálogo + las ya usadas), duración, gasto, esfuerzo RPE 1-10, notas, y desglose ejercicios->series (reps x peso) disponible en TODAS las disciplinas (con Gym/Calistenia se abre una tarjeta automáticamente). El catálogo de ejercicios se amplía solo: todo nombre usado queda como sugerencia futura.
- **Doctor:** el usuario define métricas (nombre + tipo número/sí-no/escala + unidad). Cada una registra por día, tiene gráfica y estadísticas, y se borra con confirmación (arrastra sus registros).
- **Consejos:** 10 consejos con resumen y referencia a papers reales (meta-análisis y ensayos), redactados como asociaciones poblacionales con descargo de que no sustituyen consejo médico.
- Componentes compartidos: MarcoPantalla, SelectorFecha (← Hoy →, sin futuro), CampoNumero (filtrado), CampoConSugerencias (texto libre + menú filtrado), SelectorNivel (slider), FilaHistorial + BotonBorrar (diálogo de confirmación en TODO borrado).

8. Respaldo JSON (export/import blindado)

- **Exportar:** un JSON con perfil + todas las tablas (incluidas métricas médicas), vía Storage Access Framework (el usuario elige dónde).
- **Importar** en tres pasos, sin excepciones: 1. validarRespaldo(texto): parseo + validación de rangos (edad 0-130, peso 1-500 kg, minutos 0-1440, horas HH:mm, fechas ISO, textos <=300 chars, <=100k registros). Falla con mensaje claro sin tocar la base. 2. Vista previa: diálogo con resumenRespaldo ("Se importarán: 34 de macros...") que exige confirmación. 3. aplicarRespaldo: upsert por clave; las sesiones y métricas médicas re-mapean sus ids autogenerados (los valores med.<id> se traducen al id nuevo; los huérfanos se descartan).
- Amenaza cubierta: JSON corrupto o editado que rompa la app o meta basura. Amenaza NO cubierta a propósito: el usuario falseando sus propios datos (en un tracker personal solo se engaña a sí mismo).

9. Entorno de desarrollo (esta máquina)

- **Sin Android Studio:** JDK 17 portable en C:\Users\robot\.jdk\, SDK por cmdline-tools en %LOCALAPPDATA%\Android\Sdk, compilación con .\gradlew.bat assembleDebug (APK en app/build/outputs/apk/debug/).
- **Trampa de red:** la red intercepta TLS. La JVM debe usar el almacén de certificados de Windows: ya está fijado en gradle.properties (javax.net.ssl.trustStoreType=Windows-ROOT). curl necesita --ssl-no-revoke. El sdkmanager se cuelga con descargas grandes: bajar los zips con curl y colocarlos a mano.
- **Emulador:** AVD bodymetria (Pixel 6, Android 15). Abrir Bodymetria.bat lo arranca con doble clic; Reinstalar APK.bat reinstala la última compilación. En el teléfono físico (Xiaomi), MIUI

bloquea adb install: copiar el APK a Descargas y tocarlo.

- Stack: Kotlin 2.2, Compose BOM 2025.06, AGP 8.11, Gradle 8.14.2, Room 2.7.2 (KSP), minSdk 26, targetSdk 35.

10. Decisiones y trade-offs

- **Nativo Kotlin/Compose** (vs React Native/Flutter): elegido pensando en la fase 2 (UsageStatsManager para tiempos de apps reales y Health Connect), que son APIs nativas.
- **Registro manual en v1**: el permiso de uso de apps complica la revisión de Play Store; se pospone a v1.1.
- **Base de alimentos y recetas fuera de la app** (sitio web): mantienen la app ligera; la app solo recibe los totales.
- **Tabla diario unificada**: añadir secciones de un-valor-por-día sin migraciones (así entraron micros, regla y las métricas médicas).
- **Sin cifrado del respaldo en v1** (decisión del usuario): validación + confirmación cubren integridad; la privacidad del archivo exportado queda en manos del usuario.

11. Sitio web (_{web/})

Complemento estático a la app, en React 19 + Vite (mismo espíritu: todo local, datos en localStorage, sin servidor). Enrutado con HashRouter (funciona en cualquier hosting sin reglas de redirección) y base: './'.

- **Base de alimentos** (web/src/datos/alimentos.js): valores POR RACIÓN según la Guía de Alimentos para la Población Mexicana (IMSS) — los macros son el promedio del grupo de equivalentes — más marcas comerciales por etiqueta; micronutrientes aproximados (USDA). El usuario también agrega alimentos propios capturados POR 100 g (como la etiqueta NOM-051) + gramos de la porción.
- **Perfil y metas** (datos/perfil.js): Mifflin-St Jeor x factor de actividad x ajuste por objetivo — los mismos parámetros que la app, guardados en localStorage.
- **Recetas** (datos/recetas.js): ~65 platillos (mexicanos e internacionales adaptados) + 9 postres, cada uno como lista [idAlimento, raciones]; kcal, macros y micros se CALCULAN de la base, no se inventan. El menú semanal rota solo (semilla = semana ISO) y escala cada día a la meta con un factor continuo (0.7–3) para que ningún día se pase. Ensalada cruda diaria que también rota.
- **Micros** (datos/micros.js): guía de 16 micronutrientes con rango (con dirección: mínimo, o tope para el sodio), función, evidencia científica y fuentes naturales.
- **Suplementos y Sueño**: contenido con evidencia (papers citados); la calculadora de sueño usa ciclos de ~90 min como heurístico, con el punto óptimo en 7-8 h.

12. Despliegue

- **GitHub**: repo público Eduard-Cini/bodymetria (rama main).

- **Netlify:** conectado al repo (despliegue continuo). La config vive en `netlify.toml` en la RAÍZ (Netlify la lee desde ahí): `base = "web"`, `publish = "dist"` (relativo a base), `command = "npm run build"`.
- **APK:** NO viaja en el repo (17 MB, en `.gitignore`). Se publica como **release** de GitHub (v0.1) y el botón de descarga apunta a `releases/latest/download/bodymetria.apk` (siempre la última).

13. Roadmap

- v1.1: tiempos de app automáticos (UsageStatsManager), Health Connect.
- App: sincronizar el catálogo de metas/alimentos con el del sitio.
- Play Store: keystore de release, AAB, ficha, política de privacidad, prueba cerrada de 14 días con ~12 testers.
- Análisis de mercado para decidir iOS (Compose Multiplatform reutilizaría casi todo si se decide que sí).